

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Sunday 13th September, 2026

Chapter 1

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method’s ability to consistently identify meaningful conceptual structures. The remainder of this chapter is organized as follows:

- Section 1.1 describes the software implementation of the CAR pipeline.
- Section 1.2 details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section 1.3 presents and analyses the results obtained by applying CAR and its metrics.
- Section ?? summarizes the findings and discusses their implications.

1.1 Software Implementation

For Deep Learning (DL) and Reinforcement Learning (RL), theoretical results are often difficult to apply to real-world cases. The effectiveness of methods in these fields is therefore primarily assessed through experimentation. Consequently, particular attention is given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.¹

CAR is organized as a pipeline of five stages, shown in Figure 1.1. Each stage is an independent executable, and stages communicate through artifacts written to disk. This allows each stage to be rerun or inspected independently. The parameters of each experiment are gathered in a single configuration object, described in Section 1.2. Each result can therefore be traced back to the settings that produced it. The following subsections describe the purpose of each stage, its implementation, and the libraries used.

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

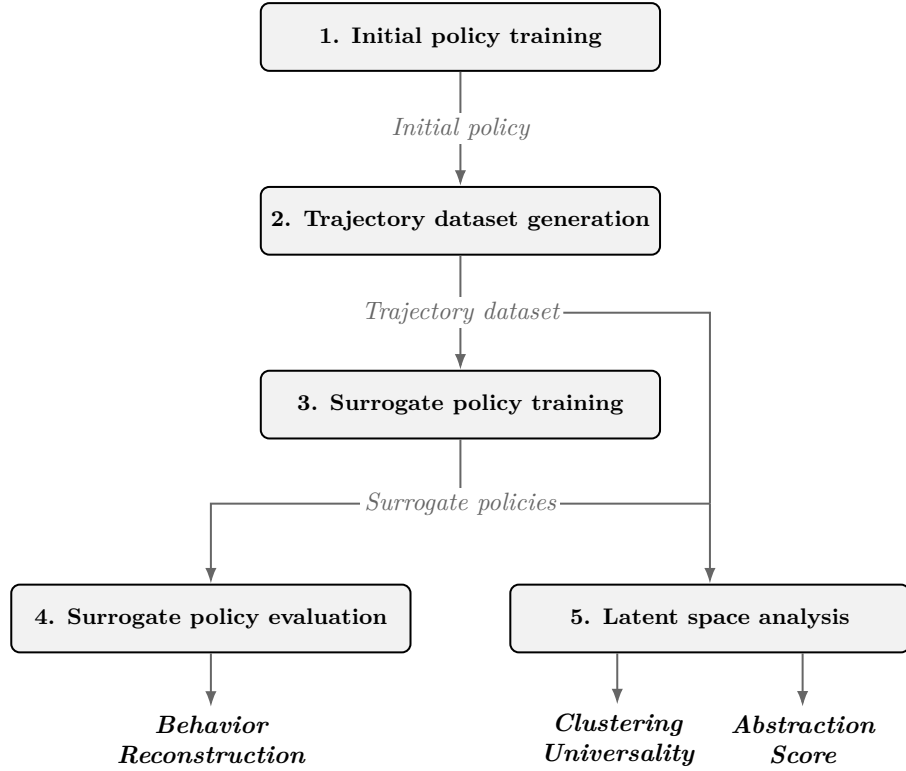


Figure 1.1: The CAR pipeline.

1.1.1 Initial policy training

The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides scalable implementations of RL algorithms and supports distributed training. The use of RLlib also allows CAR to be applied to environments implemented using the Gymnasium interface [Towers et al., 2024]. By default, the initial policy is trained using Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other RLlib algorithms, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3, can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch is the standard deep learning frameworks and is widely used for developing and training neural networks. It provides GPU acceleration and supports a wide range of neural network architectures. This makes it well suited for implementing the different policy and critic architectures used in the experiments.

The resulting policy and critic parameters are saved as a checkpoint together with the environment and training configurations.

49 1.1.2 Trajectory dataset generation

50 The second stage builds the dataset used to train the surrogate policies. RLlib
51 is used to collect trajectories in parallel using multiple environment runners. For
52 each step, the dataset stores the observation together with the parameters of
53 the action distribution produced by the initial policy.

54 Trajectories are stored as Parquet files [Apache Arrow, 2016]. Its columnar
55 structure allows efficient data access and supports concurrent processing. This is
56 useful here because the datasets contain around one million observation–action
57 pairs and cannot be loaded entirely into memory.

58 The resulting dataset is used both to train the surrogate policies and to
59 generate the latent representations analyzed by CAR.

60 1.1.3 Surrogate policy training

61 The third stage trains the surrogate policies using behavior cloning on the
62 trajectory dataset. This provides a supervised learning approach for replicating
63 the initial policy. The training procedure is implemented using PyTorch Lightning
64 [Falcon, 2019]. PyTorch Lightning provides a simplified interface for training
65 neural networks with PyTorch. It handles common training operations such as
66 data loading, optimization, checkpointing, and GPU execution.

67 Each surrogate policy is saved as a checkpoint together with the configuration
68 used for training.

69 1.1.4 Surrogate policy evaluation

70 The fourth stage evaluates how faithfully the surrogate policies reproduce the
71 behavior of the initial policy. Each surrogate is wrapped as an RLlib policy and
72 evaluated in the original environment. Its mean cumulative reward over multiple
73 episodes is compared with that of the initial policy. Evaluation episodes are
74 collected using parallel environment runners.

75 This stage computes the *Behavior Reconstruction*. It therefore verifies that
76 the surrogate policies reproduce the behavior of the initial policy sufficiently well
77 before their latent representations are analyzed. If a surrogate does not meet
78 the required level of behavior reconstruction, its training can be repeated.

79 1.1.5 Latent space analysis

80 The fifth stage performs the main analysis of CAR. The trajectory dataset
81 and trained surrogate policies are used to generate the latent representations.
82 These representations form the point clouds that are analyzed using clustering
83 algorithms.

84 The clustering step relies on the cuML library [RAPIDS, 2023], which pro-
85 vides GPU-accelerated clustering algorithms. This reduces the computational
86 cost and allows point clouds containing tens of thousands of representations to
87 be analyzed.

88 The resulting partitions are used to compute the *Clustering Universality*
89 and the *Abstraction Score*. The stage also produces the material required for
90 the qualitative analysis, including the clustered point clouds and the cluster
91 assignment of each observation.

92 Table 1.1 summarizes the main third-party libraries used in the pipeline.

Pipeline Stage	Library	Description
Initial policy training	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed RL library used to train the initial policy.
	PyTorch [Paszke et al., 2019]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [Towers et al., 2024]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [Apache Arrow, 2016]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [Falcon, 2019]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [RAPIDS, 2023]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 1.1: Main software libraries used at each stage of the CAR pipeline.

1.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architecture of the policies, and the main parameters of each pipeline stage.

1.2.1 Environments

All experiments presented in this manuscript use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages. First, Atari is a widely used benchmark in the RL community, with many studies using these environments as standard evaluation settings. This makes the experimental results easier to compare with existing work. Second, these environments were not specifically designed for RL, which reduces the risk of introducing biases related to the design of the environments themselves. Finally, the benchmark provides a diverse set of well-known environments with varying levels of complexity. This diversity makes it possible to evaluate CAR across different types of tasks and levels of difficulty.

The configuration of the Atari environments follows common practice in RL research [Machado et al., 2017].

In each environment, the agent receives visual observations in the form of raw pixels. Each image is resized to 64×64 pixels and converted to grayscale to reduce memory requirements. An exception is made for Pac-Man, where color information is important for determining the behavior of other agents in the environment. The original color information is therefore preserved for this environment.

To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting

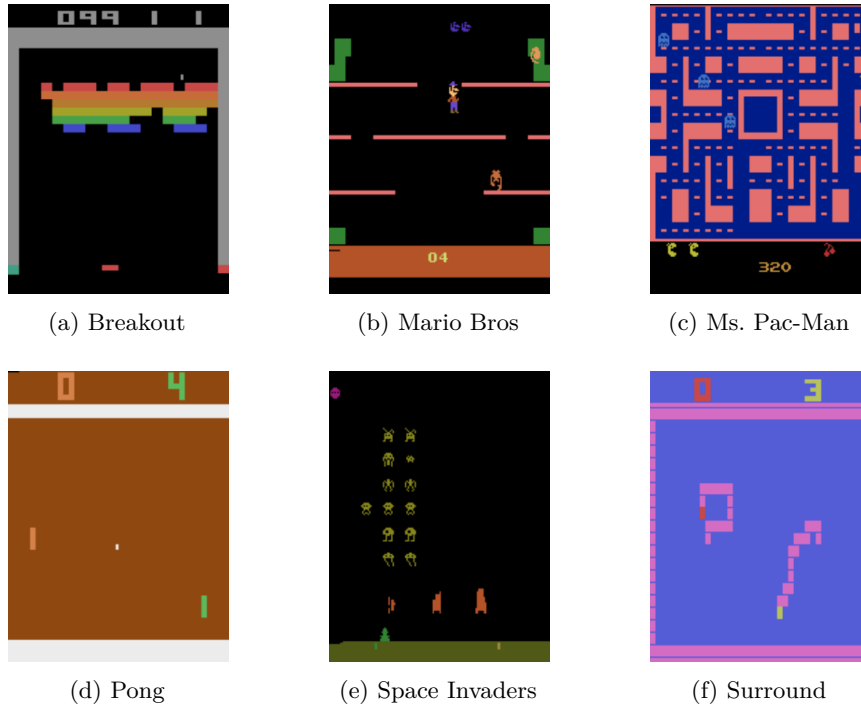


Figure 1.2: The six Atari environments used to evaluate CAR.

observation has a shape of $64 \times 64 \times 4$. This allows the agent to infer the motion of objects from successive observations rather than relying on a single frame.

Frame skipping is also used to reduce the duration of each episode. The agent selects a new action every five frames, while the selected action is repeated for the intervening frames. This provides sufficient temporal resolution for effective control while reducing the number of decisions required during an episode.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete.

Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

With the environment configuration established, the objective of each environment is described below. Figure ?? provides an overview of the six environments used in the experiments.

Breakout. The agent moves a paddle at the bottom of the screen (Figure 1.2a) to bounce a ball into a wall of bricks, each broken brick scoring a point, and the ball speeding up as the wall is cleared.

Mario Bros. A single-screen platform game (Figure 1.2b). The player clears each phase by hitting enemies from below to flip them, then kicking them off

the platforms before they recover. A central POW block flips every enemy at once but can only be used a few times per phase.

Ms. Pac-Man. The player moves through a maze (Figure 1.2c), eating pellets while avoiding four ghosts, with power pellets briefly making the ghosts edible. The maze structure and the many possible local situations, such as a nearby ghost, an intersection, or an active power pellet, produce a large number of distinguishable states.

Pong. A minimal two-player game (Figure 1.2d) in which the agent controls one paddle and must send the ball past the opponent.

Space Invaders. The agent moves a cannon along the bottom of the screen (Figure 1.2e), firing at descending rows of aliens while dodging their shots and using destructible shields, with a bonus saucer occasionally crossing the top.

Surround. A two-player game on a grid (Figure 1.2f). Each player leaves a solid trail behind a moving cursor, and the goal is to force the opponent to crash into a trail or a wall.

1.2.2 Neural architecture

The surrogate policies use a convolutional neural network to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using 4×4 kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a multilayer perceptron that produces the parameters of the action distribution.

The architecture of the multilayer perceptron varies across environments. For Pong and Space Invaders, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. Figure 1.3 illustrates both configurations. The size of the output layer matches the number of actions available in each environment: 4 for Breakout, 18 for Mario Bros, 9 for Ms. Pac-Man, 6 for Pong, 6 for Space Invaders, and 5 for Surround.

1.2.3 Parameters

Having defined the environments and neural architectures, the remaining parameters of the CAR pipeline must now be specified. The main parameters shared across the experiments are summarized in Table 1.2. There are too many parameters to discuss them all in detail, as each stage of the pipeline introduces its own configuration choices.

Given the large number of possible configurations, it is not feasible to evaluate all possible combinations. The choices made in this work are therefore based primarily on standard practices from the literature and on practical experience. Whenever possible, we used the simplest suitable configuration. For the clustering algorithm and distance metric in particular, the simplest suitable choices are used to establish a baseline.

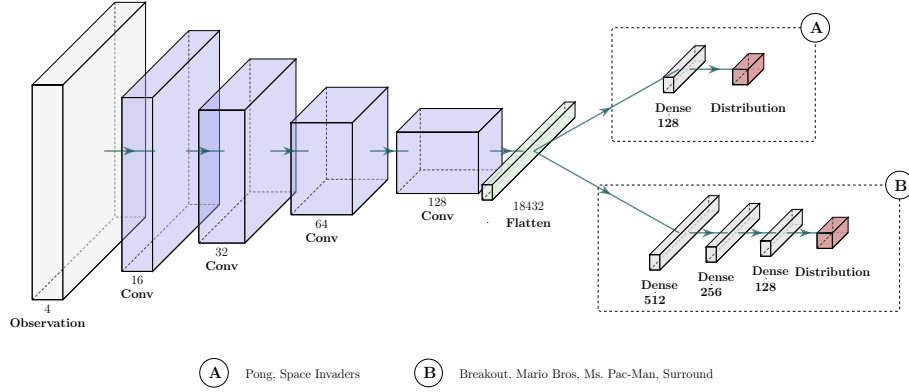


Figure 1.3: Architecture of the surrogate policies.

181 These choices do not restrict the CAR methodology itself. Most components
 182 can be replaced or configured differently depending on the application. The
 183 evaluation metrics provide an objective basis for comparing these alternatives and
 184 determining which configurations produce the most consistent and meaningful
 185 results.

186 1.3 Results

187 This section reports the results of applying CAR to the six environments. Sec-
 188 tion 1.3.1 verifies that the initial policies reach a competent level of play, while
 189 Section 1.3.2 evaluates whether the surrogate policies faithfully reproduce their
 190 behavior using the *Behavior Reconstruction*. Section 1.3.3 analyzes the latent
 191 representations using the *Abstraction Score* and *Clustering Universality*.

192 1.3.1 Initial policy training

193 Figure 1.4a shows how the initial policies improve during PPO training. For
 194 each environment, the mean episode return is normalized so that 0% and 100%
 195 correspond to the lowest and highest mean return observed during that run. In
 196 every environment the return increases and then reaches a plateau, at which
 197 point training is stopped. The number of iterations required to reach this plateau
 198 varies from a few hundred, for Pong, to several thousand, for Surround and Ms.
 199 Pac-Man. For Ms. Pac-Man the return decreases after its peak. Since the best
 200 checkpoint is retained rather than the last one, this does not affect the policy
 201 that is subsequently analyzed.

202 To confirm that the trained policies play competently, their scores are com-
 203 pared with the reference scores reported by [Machado et al., 2017]. Each initial
 204 policy reaches a score of the same order as these references, so it constitutes a
 205 reasonable target for imitation.

206 1.3.2 Surrogate policy training

207 Figure 1.4b shows, for each environment, the training action loss of the three
 208 surrogate policies. In every case the loss decreases smoothly, and the three

Environment configuration	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Initial policy training	
Algorithm	PPO
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Batch size	2048
Optimizer	Adam
Learning rate	1×10^{-4}
Latent space analysis	
Embedding per surrogate policy	50 000
Clustering algorithm	k -means
Distance	Euclidean distance
Number of clusters K	2 to 30

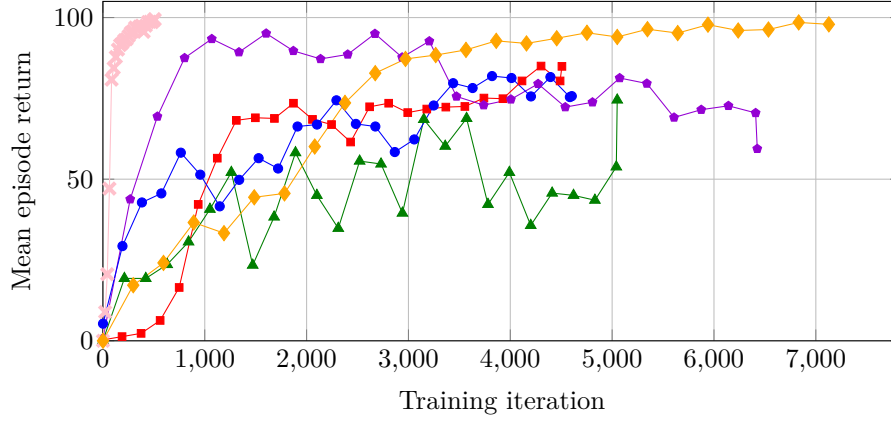
Table 1.2: Main experimental parameters.

independently trained policies follow almost identical trajectories, converging to the same value.

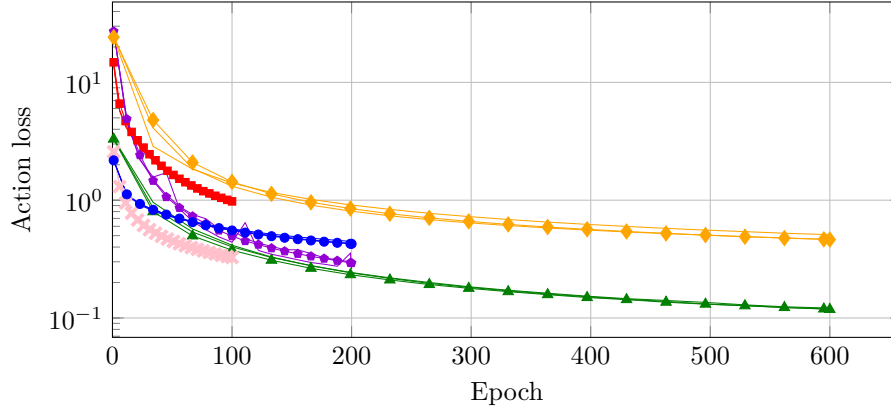
We further evaluate the ability of the surrogate policies to replicate the behavior of the initial policies using the *Behavior Reconstruction* (Table 1.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for Breakout to 1.00 for Ms. Pac-Man and Surround. They therefore reproduce the decisions of the initial policies closely enough for their latent representations to be analyzed in place of those of the initial policies.

1.3.3 Latent space analysis

With the behavioral fidelity of the surrogate policies established, this section analyzes their latent representations using the *Abstraction Score* and *Clustering Universality*, also reported in Table 1.3.



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.

—▲ Breakout —■ Mario Bros —● Ms. Pac-Man
 —× Pong —● Space Invaders —◆ Surround

Figure 1.4: Training of the initial and surrogate policies.

222 For the remainder of the analysis, a random subset of 50,000 observations
 223 is embedded into the Latent Space (LS) of each surrogate policy, using the
 224 representation extracted directly after the last convolutional layer, and the
 225 resulting point cloud $\mathcal{E}$ is partitioned with k -means clustering [MacQueen, 1967].

226 Figure 1.5a reports the evolution of *Clustering Universality* with respect to
 227 the number of clusters. After the first few values of K , every curve settles onto
 228 a plateau whose level depends on the environment, from about 0.45–0.50 for
 229 Breakout and Pong to about 0.65 for Ms. Pac-Man and Space Invaders. Since
 230 *Clustering Universality* is computed from the Adjusted Rand Index (ARI), and an
 231 ARI of 0.5 already corresponds to substantial agreement [Amezquita et al., 2020],
 232 these values indicate that independently trained surrogate policies recover a

Environment	<i>Behavior Reconstruction</i>	<i>Abstraction Score</i>	<i>Clustering Universality</i>
Breakout	0.83 $\pm$ 0.03	0.82 $\pm$ 0.07	0.48 $\pm$ 0.11
Mario Bros	0.99 $\pm$ 0.03	0.71 $\pm$ 0.14	0.55 $\pm$ 0.10
Ms. Pac-Man	1.00 $\pm$ 0.03	0.43 $\pm$ 0.09	0.65 $\pm$ 0.08
Pong	0.95 $\pm$ 0.01	0.99 $\pm$ 0.00	0.49 $\pm$ 0.05
Space Invaders	0.97 $\pm$ 0.07	0.69 $\pm$ 0.06	0.67 $\pm$ 0.07
Surround	1.00 $\pm$ 0.01	0.55 $\pm$ 0.07	0.61 $\pm$ 0.09

Table 1.3: Evaluation of *Behavior Reconstruction*, *Abstraction Score* and *Clustering Universality* across the six Atari environments.

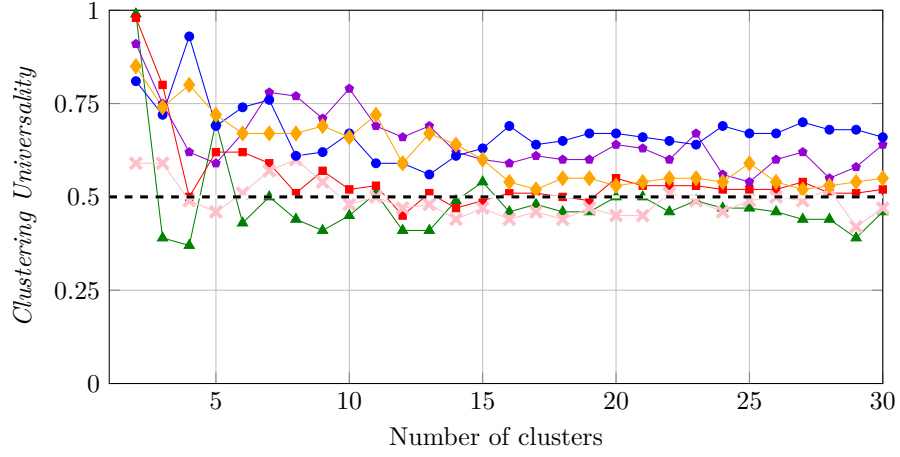
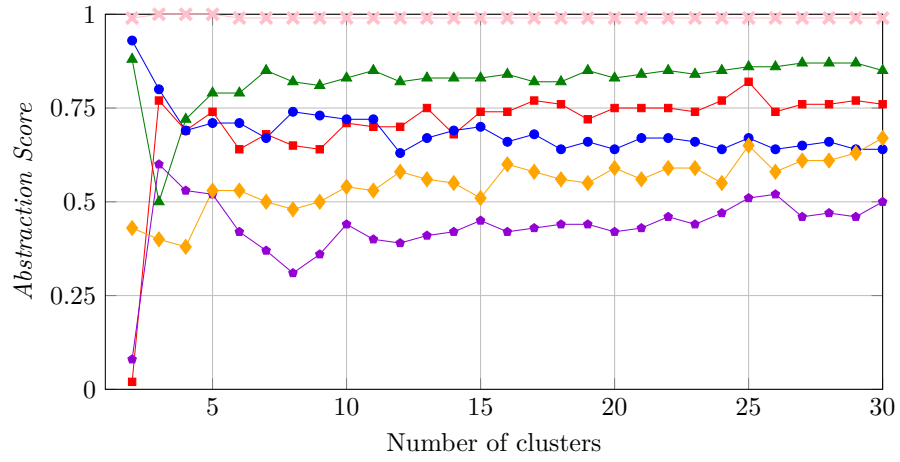
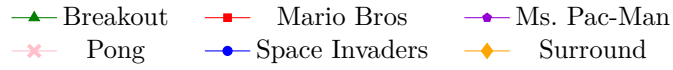
similar cluster structure in every environment, and that this structure is stable with respect to the number of clusters. The strength of this agreement varies across environments, but remains clearly above chance everywhere. The absence of a single optimal number of clusters suggests that K should be chosen according to the level of granularity desired for the analysis.

The *Abstraction Score*, in contrast, varies markedly across environments. It is close to 1 for Pong (0.99) and high for Breakout, Mario Bros and Space Invaders (0.69 to 0.82): in these environments the latent clusters are largely independent of a direct clustering of the observations or of the action distributions. It is lower for Surround (0.55) and Ms. Pac-Man (0.43). In these two environments the position of the agent on the screen largely determines both the raw observation and the decision to take, so a clustering of the LS naturally coincides with a clustering of the observations, and the *Abstraction Score* cannot separate the two; a low score does not by itself imply that the representation is not abstract.

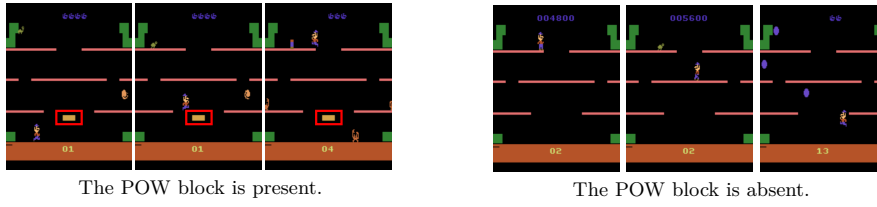
With fewer clusters, CAR highlights broad strategic patterns. For instance, in the Mario Bros environment, the two main groups correspond to whether the POW block is present (highlighted in red in Figure 1.6A) or not. This is particularly interesting, as the use of the POW block is a key determinant of strategy in this environment [AtariAge, 2024].

With a larger number of clusters, the analysis captures more fine-grained tactical behaviors, such as “Attacking the left” and “Navigating through the level” (see Figure 1.6B).

Both perspectives are valid, depending on whether the goal is to study strategic or tactical behaviors. Similar interpretable structures can be observed across the evaluated environments.

(a) Evolution of *Clustering Universality* with respect to the number of clusters.(b) Evolution of *Abstraction Score* with respect to the number of clusters.Figure 1.5: Evolution of *Clustering Universality* and *Abstraction Score* with respect to the number of clusters.

(A) The case with 2 clusters.



(B) The case with 5 clusters.

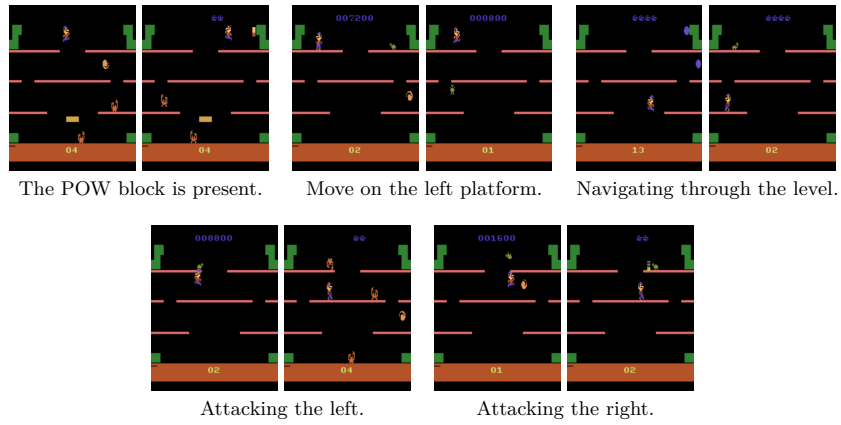


Figure 1.6: Effect of the number of clusters on the groupings identified by CAR in the Mario Bros environment.

Acronyms

APPO Asynchronous Proximal Policy Optimization 2

ARI Adjusted Rand Index 9

CAR Clustering Agent Representation 1–7, 10, 12

DL Deep Learning 1

DQN Deep Q-Networks 2

GPU Graphics Processing Unit 4

LS Latent Space 9, 10

PPO Proximal Policy Optimization 2, 7–9

RL Reinforcement Learning 1, 2, 4

Bibliography

- [Amezquita et al., 2020] Amezquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [AtariAge, 2024] AtariAge (2024). Mario bros. atari 2600. **Web**.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

- 301 [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R.,
302 Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018).
303 Ray: A distributed framework for emerging AI applications. In 13th USENIX
304 Symposium on Operating Systems Design and Implementation (OSDI), pages
305 561–577.
- 306 [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J.,
307 Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison,
308 A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy,
309 S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An
310 imperative style, high-performance deep learning library. In Advances in
311 Neural Information Processing Systems (NeurIPS).
- 312 [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine
313 learning. <https://github.com/rapidsai/cuml>.
- 314 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
315 Klimov, O. (2017). Proximal policy optimization algorithms.
- 316 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
317 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
318 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
319 environments.